Project Proposal: Modeling Malware Concept Drift via Continuous-Time Neural ODEs

1. Background and Motivation

Machine-learned malware classifiers routinely score above 0.99 F1 in static, cross-validated evaluations, yet post-deployment performance can degrade by as much as 50% within months. This is not a symptom of poor tuning — it is concept drift: attackers continuously mutate code, adopt new packers, and change delivery infrastructure, so the statistical relationship between a binary's features and its true label shifts over time. Standard k-fold evaluation hides this problem entirely, because randomly splitting a chronologically evolving dataset leaks future malware variants into training, producing an illusion of robustness that the TESSERACT framework (Pendlebury et al., 2019) exposed as a pervasive spatio-temporal bias in the field.

Two existing tools fall short of fixing this. Static classifiers (Random Forests, XGBoost, MLPs) freeze a decision boundary at training time and have no mechanism for tracking how malware clusters migrate through feature space. Discrete sequence models (RNNs, LSTMs, GRUs) add some temporal awareness but assume fixed, regularly spaced time steps — an assumption malware sampling violates constantly, since a threat actor may release a dozen variants in a week, then go quiet for months. Neither approach represents drift as what it actually is: a continuous process happening between observations.

This project proposes to model malware evolution directly as a continuous-time dynamical system using a Latent Neural ODE. This reframes drift-resilient detection

from a periodic-retraining problem into a forecasting problem: instead of a boundary that goes stale, the model learns the rules governing how the boundary moves. As a secondary extension — pursued only once the core model is validated and stable — we plan to test whether augmenting it with a discrete jump mechanism improves handling of sudden, discontinuous shifts (e.g., a zero-day disclosure or a geopolitical shock). This project is scoped in two explicit tiers throughout: a core deliverable we are committing to, and a stretch extension we will attempt if time and stability allow.

2. Clearly Defined Problem Statement

1. **Core:** Build and rigorously evaluate a Latent Neural ODE classifier for Windows PE malware that models concept drift as a continuous vector field, and demonstrate — under strict chronological (TESSERACT-style) evaluation — that it degrades significantly less over time than static baselines, while remaining trainable on a single consumer-grade GPU.
2. **Stretch:** If the core model is stable, extend it with a discrete jump network and evaluate whether it improves classification stability through an abrupt-drift window, compared to the non-jump Latent ODE.

3. Objectives of the Project

Core Objectives (Committed)

•

Technical Objective	Implementation Strategy	Current Baseline	Target Metric
Temporal drift resilience	Latent Neural ODE with continuous vector field, trained via the adjoint sensitivity method	Static XGBoost / LightGBM trained on early months; documented AUROC decay of up to ~50%	AUT trajectory stays within ~10% of initial AUROC
Class-imbalance robustness	Focal Loss (tuned focusing parameter γ) in place of cross-entropy	Cross-entropy baseline	Macro-averaged F1 on malware subsets ≥ baseline

Technical Objective	Implementation Strategy	Current Baseline	Target Metric
		over-weights easy benign samples	
Computational feasibility	Adjoint sensitivity method for O(1)-memory backpropagation	Naive backprop requires O(N) memory, infeasible on consumer hardware	Full training completes on a single RTX 4090 GPU

Stretch Objectives (Contingent)

Technical Objective	Implementation Strategy	Current Baseline	Target Metric
Discontinuous / sudden-drift handling	Stochastic jump network that resets the latent state on OOD observations	Standard Latent ODE bound by Picard–Lindelöf theorem	Stable classification through abrupt-drift window

4. Proposed Dataset & Access Source

Dataset: BODMAS (Blue Hexagon Open Dataset for Malware Analysis) — 134,435 PE samples collected over a contiguous 14-month window (Aug 2019–Sep 2020). Features include 2,381-dimensional vectors spanning PE metadata and histograms.

- Why BODMAS over alternatives: EMBER (2018) and SOREL-20M offer larger scale (SOREL: ~20M samples) but lack BODMAS's temporal granularity; per TESSERACT-style findings, scale alone does not mitigate drift if evaluation windows aren't strictly chronological.

Data Partitioning (Chronological):

Train: Months 0–7 (Aug 2019–Mar 2020)
Validation: Months 8–9 (Apr–May 2020)
Test: Months 10–13 (Jun–Sep 2020)

- Validation — Months 8–9 (Apr–May 2020), used only for hyperparameter tuning and ODE solver tolerance calibration

- Test — Months 10–13 (Jun–Sep 2020)
- Dataset is publicly downloadable (Yang et al., 2021 / BODMAS GitHub) with no access restrictions, rate limits, or PII/GDPR exposure — features are structural binary metadata, not personal data.
- Natural experiment: The March 2020 boundary between training and validation coincides with the onset of COVID-19-themed malware campaigns, providing a real, documented sudden-drift event — this is what the stretch-goal jump mechanism (Section 3) would be tested against, if reached.
- Preprocessing: An encoder network compresses the 2,381-dim LIEF feature vector into a compact latent space before ODE integration; no additional augmentation is needed since features are already numerically pre-extracted.

5. Proposed Deep Learning Methodology

Core Architecture: Latent Neural ODE

> **Encoder:** Maps feature vectors into low-dimensional latent state $z(t_0)$.
> **Continuous Integration:** Evolves state via $dz(t)/dt = f(z(t), t; \theta)$ using adaptive-step solvers.
> **Decoder:** Maps evolved latent state to final classification logit.

- Encoder: maps the 2,381-dim PE feature vector into a low-dimensional latent state $z(t_0)$.
- Continuous integration: the latent state evolves as $dz(t)/dt = f(z(t), t; \theta)$, integrated forward with an adaptive-step solver (Dormand–Prince / dopri5) to project the state to a future time.
- Decoder: a standard MLP maps the evolved latent state to the benign/malicious logit.
- Planned Extension (stretch, time-permitting): Jump Network. On detection of a highly anomalous (out-of-distribution) sample, the trajectory would be reset via a separate network, $z(\tau^+) = h(z(\tau^-), x_\tau; \varphi)$, bypassing the continuous flow to absorb sudden shifts. This is scoped as an addition layered onto a working core model, not a prerequisite for it — see Section 8 for when it is attempted.
- Optimization techniques (core): Adjoint sensitivity method for constant-memory backpropagation through the solver; Focal Loss to counter class imbalance; kinetic/Jacobian regularization to keep the learned vector field smooth and control the Number-of-Function-Evaluations (NFE).

- Baselines for comparison (core): XGBoost, LightGBM, and a parameter-matched MLP (all static), trained on identical data splits.
- Technology stack: PyTorch with a Neural-ODE library (e.g., torchdiffeq) for the solver and adjoint method; scikit-learn / xgboost / lightgbm for baselines.
- Hardware: Single consumer GPU (RTX 4090) or Colab Pro as primary target, given the adjoint method's constant memory footprint; university cluster as fallback for ablation sweeps.
- Fallback methodology: If the core Latent ODE struggles with high-dimensional virtual drift (see Section 6 risk mitigation), pivot to a Continuous Normalizing Flow (CNF) with flow-matching training as an alternative core model, rather than layering it onto the ODE.

6. Feasibility & Risk Mitigation

- Data feasibility: BODMAS is a public, static-download academic dataset — no API rate limits, no PII, no GDPR exposure (structural PE metadata only).
- Computational feasibility: The adjoint method keeps backward-pass memory constant regardless of solver steps, making training tractable on a single consumer GPU; baseline models (XGBoost/LightGBM/MLP) are inexpensive by comparison. Estimated training time should be scoped once solver step-size behavior is profiled in Week 1–2 (see schedule).
- Deployment feasibility: This is a research/evaluation proposal, not a production deployment; the deliverable is a set of quantitative results (AUT curves, tables) and a written report, not a production or demonstration interface.
- Primary technical risk — NFE explosion / stiff ODEs: As the learned vector field grows more contorted to separate increasingly sophisticated malware, the ODE can become numerically stiff, forcing adaptive solvers like dopri5 to take vanishingly small steps and causing training time (and adjoint-method numerical error) to blow up.Mitigation: monitor solver step counts during training; if stiffness is detected, switch from explicit (dopri5) to an implicit single-step solver; apply kinetic/Jacobian regularization to keep the vector field smooth.
- Plan B: if core Latent ODE training remains unstable after mitigation, pivot to the CNF/flow-matching alternative described in Section 5, or fall back to a periodically-retrained static baseline as a floor comparison.
- Secondary risk — stretch goal contention: the jump network (Section 3) depends on the core model being stable well ahead of the semester's end. If Weeks 1–11

run over schedule, the jump extension is dropped without affecting the core deliverable or grading criteria, since it was never a dependency of the primary claim.

7. Expected Technical Deliverables

- Minimum Deliverable (core, committed):
- Static baseline profiling (XGBoost/LightGBM/MLP) to quantify raw drift magnitude.
- Latent ODE temporal integration and evaluation on the same held-out future months.
- Longitudinal drift quantification via the Area Under Time (AUT) metric — AUROC plotted month-by-month rather than as a single aggregate score.
- A written report and result plots comparing all models under identical chronological splits.
- Extended Deliverable (stretch, time-permitting):
- Jump network extension and evaluation across the March 2020 sudden-drift window (Section 4).
- Architectural ablations (dopri5 vs. fixed-step rk4; decoupled encoder/decoder without the ODE) to isolate what the continuous integration actually contributes.
- Latent-space visualization via UMAP, comparing initial vs. chronologically-evolved embeddings.
- Framing of the contribution: Most prior work treats drift as a retraining problem — periodically re-labeling and re-fitting a static boundary. This project instead tests whether adversarial evolution can be usefully modeled as a continuous process the network learns directly, rather than a boundary requiring periodic re-fitting — with sudden-shift handling investigated as a planned, clearly-scoped extension once the core continuous-time claim is established.
-

8. Project Schedule & References

- Duration: ~16-week single-semester scope, split into a committed core phase and a contingent extension phase.
-

Weeks	Milestone	Tier
1–2	Data acquisition & preprocessing; build chronological splits.	Core
3–4	Static baseline training & profiling (XGBoost, MLP).	Core
5–9	Latent ODE implementation & adjoint method profiling.	Core
10–11	Longitudinal AUT evaluation & baseline comparison.	Core
12–13	Final results consolidation and report writing.	Core
14–16	Jump network extension & architectural ablations.	Stretch

- Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. (2018). Neural Ordinary Differential Equations. NeurIPS. https://arxiv.org/abs/1806.07366
- Rubanova, Y., Chen, R. T. Q., & Duvenaud, D. (2019). Latent Ordinary Differential Equations for Irregularly-Sampled Time Series. NeurIPS. https://proceedings.neurips.cc/paper/2019/hash/42a6845a557bef704ad8ac9cb4461d43-Abstract.html
- Herrera, C., Krach, F., & Teichmann, J. (2020). Neural Jump Ordinary Differential Equations: Consistent Continuous-Time Prediction and Filtering. arXiv:2006.04727. https://arxiv.org/pdf/2006.04727
- Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., & Cavallaro, L. (2019). TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time. USENIX Security. https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury
- Yang, L., Ciptadi, A., Laziuk, I., Ahmadzadeh, A., & Wang, G. (2021). BODMAS: An Open Dataset for Learning-based Temporal Analysis of PE Malware. DLS. https://gangw.cs.illinois.edu/DLS21_BODMAS.pdf
- BODMAS Dataset repository: https://github.com/whyisyoung/BODMAS